NILECARE

Negative Testing & Reconnaissance Report

Non-exploitable test outcomes, attack-surface mapping and route discovery

Target: NileCare / Egyptian Specialist Hospital patient portal

Assessment window: 28–29 October 2026 (as recorded in the source notes)

Application profile: Flask + Jinja2, server-rendered, Cloudflare-fronted

Assessment type: Authorized grey-box/self-assessment of a shared lab instance

Report purpose
This document records reconnaissance and test cases for which the team notes did not establish an additional exploitable condition. Confirmed vulnerabilities already documented in the separate NileCare Security Findings document have deliberately been excluded from the findings narrative.

Prepared from the supplied AI-REPORTS.md and NileCare_Security_Findings_Summary.md. No new live testing was performed for this report.

1. Executive Summary

This report isolates the assessment work that did not produce a separately validated exploit within the recorded test scope. The most substantial negative result is the absence of a demonstrated remote-code-execution (RCE) path. The team also recorded unsuccessful tests for server-side template injection (SSTI), SQL injection, command injection, unsafe deserialization, path traversal/local file inclusion, debug-console exposure, source/configuration disclosure, upload-to-executable-path bypasses, CORS misconfiguration, SSRF, HTTP method tampering, host-header poisoning, parameter pollution and mass assignment.

The hidden admin route was reachable, but the notes record no authentication bypass, no accessible post-login administrative surface, and no successful injection. Several guessed sub-routes and alternate admin paths returned 404. This supports a limited conclusion about the tested paths and payloads—not a claim that the entire application is secure.

The report also records route and resource discovery. The source notes state that robots.txt listed 100 decoy-style paths, with one live admin route; however, the complete 100-entry list was not preserved in the supplied material. Only the paths explicitly present in the notes are listed here.

Important distinction: the separate NileCare findings document contains confirmed issues involving stored XSS, IDOR/broken access control, information disclosure, client-controlled discounts and password-reset abuse. Those are not repeated as findings here. Upload validation is discussed only in the context of additional bypass attempts that were rejected or served as inert static content.

2. Scope, Inputs and Limitations

Item	Recorded detail
Target	https://tan-clinic-mono-flood.trycloudflare.com
Application	NileCare / Egyptian Specialist Hospital patient portal
Stack indicators	Flask, Jinja2, server-rendered pages, Cloudflare front end
Dates	28–29 October 2026; a later re-test is also described in the notes
Method	Grey-box/authenticated testing, route probing, request manipulation, payload testing and response comparison
Evidence location	The source notes refer to /tmp/opencode/tanclinic/ with recon/, evidence/, raw HTTP bodies and fuzz JSON. Those artifacts were not separately supplied for this report.

This report is a synthesis of the two supplied Markdown documents only. It does not independently verify the target, reproduce requests, inspect the original evidence directory, or establish the current state of the live application. Findings described as negative mean no exploit was demonstrated by the team using the recorded tests.

3. Reconnaissance and Surface Discovery

The recorded reconnaissance covered response headers, TLS, robots.txt, technology fingerprinting, route probing, directory/API discovery, static assets and authenticated patient-page mapping. The team logged in with a provided patient test account and crawled patient pages and forms. Credentials are intentionally omitted from this report.

3.1 Explicitly identified application routes

Path	Why it appears in the notes	Reporting treatment
`/admin_portal_temp_secure_9203`	Hidden admin login path; listed by robots.txt; reachable login form.	Included in negative admin-surface tests below.

Path	Why it appears in the notes	Reporting treatment
`/dashboard`	Authenticated patient landing page and post-login destination.	Mapped as an authenticated page; no separate negative exploit claim.
`/profile`	Profile form encountered during authenticated mapping.	Excluded from detailed results because the source describes successful sensitive account changes.
`/console`	Checked as a possible Werkzeug/debug console.	404 recorded; included as negative exposure test.
`/logout`	Logout route checked using GET and POST.	No security claim here; session behavior was a positive issue and is excluded.
`/static/uploads/attachments/`	Upload storage path observed during upload tests.	Directory listing not enabled; file serving behavior tested.
`/robots.txt`	Disclosed a list of candidate paths, including the admin path.	Discovery source; see path-discovery section.

3.2 Other endpoints observed during mapping

The AI notes explicitly name `/transfer_points`, `/reports/create`, `/reports/upload-attachment`, `/doctors/review`, `/forgot-password`, `/reset-password`, and `/api/v1/reports/view`. These endpoints are recorded here only as discovered surface. The supplied NileCare findings document and AI notes describe confirmed issues on several of them; therefore they are not presented as negative or secure outcomes in this report.

3.3 robots.txt and candidate directories

The notes report approximately 100 randomly suffixed decoy paths in robots.txt, with exactly one path resolving successfully: `/admin_portal_temp_secure_9203`. The other 99 reportedly returned 404. Example decoy entries preserved in the notes are:

- `/admin_private_95466`
- `/api_dump_13280`
- `/auth_restrict_1851`

> The complete robots.txt path list is not included in the supplied documents. Accordingly, this report does not invent or reconstruct the remaining entries.

3.4 Admin route and alternate path probing

The following admin-related paths were explicitly tested and recorded as 404: `/admin`, `/administrator`, `/manage`, `/system`, and `/wp-admin`. The following candidate sub-routes under the hidden admin portal were also reported as 404: `/dashboard`, `/users`, `/settings`, `/logs`, `/api`, `/console`, `/export`, `/backup`, and `/panel`.

4. Negative Test Results

The table below consolidates tests explicitly described in the AI report as unsuccessful, rejected, not reachable or not demonstrated. These are test outcomes, not guarantees of security.

Test area	What the team tried	Recorded outcome
Jinja2 SSTI	Arithmetic/configuration-style template expressions in report title, body, lab and profile-name inputs.	Payloads rendered literally; no template evaluation demonstrated.
Unsafe deserialization	Malformed/crafted pickle-style or serialized token inputs.	Rejected as invalid token format; notes cite strict JSON parsing for the tested token path.
SQL injection	Login and other fields; admin login; report-related values; boolean/time-style probes.	No exploitable SQLi path or repeatable boolean/time differential identified; notes state parameterized handling.
Command injection	Shell metacharacters and timing payloads in name/date/discount-code style inputs.	No repeatable delay or output difference; no command execution demonstrated.
Upload executable extension bypass	.php, .phtml, .html, .svg, .jpg.php, .php.gif and .php.jpg-style filenames.	Rejected or renamed; allowed image formats enforced in the tested cases.
Magic-byte spoofing	A script-like payload with a PNG signature/header.	Rejected with an invalid file magic-bytes signature message.
Upload execution	Requested a stored file named foo.html.png.	Returned HTTP 200 with image/png content type; treated as static image content, not executed as HTML.
Path traversal in upload filename	Traversal-style filenames targeting templates and relative paths.	Basename reportedly stripped; file written under the attachments upload directory.
Directory listing	Requested the attachments directory.	Directory listing not enabled.
Path traversal / LFI	Traversal and local-file-read attempts against application paths.	No successful read outside intended application paths recorded.
Debug console	Checked /console for Werkzeug/debug console exposure.	404; debug console not reachable.
Source/config disclosure	Checked common source, environment, VCS, backup and metadata paths.	All tested paths returned 404; no sensitive file disclosure demonstrated.
Flask secret-key guessing	Candidate secret-key guesses and session-forgery attempts.	No forged session accepted; the notes report 30,000 guesses in one pass and 38 candidates in a later pass.
CORS	Cross-origin Origin header with credentials.	No Access-Control-* headers returned in the described tests.
SSRF	Looked for user-controlled outbound-request functionality.	No outbound request sink identified; no SSRF demonstrated.
HTTP method tampering	PUT, PATCH, DELETE and TRACE across tested routes.	405 responses recorded; OPTIONS handled by Flask default.

Test area	What the team tried	Recorded outcome
Mass assignment	role, is_admin, user_id, balance and points fields during registration.	Fields reportedly ignored; no privilege/role mutation demonstrated.
Host header poisoning	Host and X-Forwarded-Host manipulation against reset-link generation.	Reset links were relative in the observed behavior; poisoning not demonstrated.
Parameter pollution	Duplicate parameter values.	First value reportedly selected consistently; no bypass demonstrated.

5. Admin Portal: Tested Surface and Verdict

The hidden route `/admin_portal_temp_secure_9203` presented a standalone login form accepting email and password. The team tested login handling, request formats, headers, patient-session reuse, mass-assignment parameters, guessed sub-routes and alternate admin paths.

- SQL injection payloads in email/password did not change the observed response in a way that demonstrated injection.
- Empty or omitted passwords were rejected.
- JSON and array-style request bodies were rejected.
- Header spoofing attempts using X-Forwarded-For, X-Real-IP, X-Original-URL, X-Rewrite-URL, X-Admin and X-User-Role had no observed effect.
- Reusing a patient session cookie did not authenticate to the admin portal.
- Mass-assignment parameters such as role=admin and is_admin=1 had no observed effect.
- The listed guest sub-routes and alternate admin paths returned 404.
- A dictionary attempt of 4,053 passwords produced no successful login; the notes record no lockout/throttling during the attempt.

Verdict: no admin authentication bypass, injection or reachable post-auth administrative surface was demonstrated by the recorded tests. The notes characterize the route as isolated or honeypot-like. That characterization is an assessment hypothesis, not proof of the route's intended purpose. The admin login also had a separately documented account-enumeration/rate-limiting weakness; that positive issue is deliberately excluded from this negative-testing report.

6. RCE Assessment

No reliable remote-code-execution path was identified across the reachable surfaces tested. The negative result was supported by the following categories of probes:

- SSTI probes in multiple user-controlled text fields rendered literally.
- Deserialization payloads were rejected as invalid token formats.
- Command-injection timing/output probes did not produce a repeatable timing delta or output change.
- Executable upload extensions and spoofed file signatures were rejected in the tested cases.
- Stored files tested through the attachment path were served as image/png rather than interpreted as executable HTML.
- Traversal/LFI attempts did not retrieve files outside intended application paths.
- The debug console path was not reachable.

Conclusion: RCE was not achieved and no RCE vector was validated during this assessment. This is bounded by the payloads, inputs, routes and target state actually tested; it is not a claim that RCE is impossible.

7. Additional Protocol and Application Checks

The source notes also record the following outcomes:

Area	Recorded observation	Interpretation
HTTP methods	PUT, PATCH, DELETE and TRACE returned 405 on tested routes; OPTIONS used Flask default handling.	No method-override bypass demonstrated.
CORS	Origin: https://evil.example with credentials did not produce Access-Control-* headers.	No permissive CORS behavior demonstrated in those requests.
SSRF	No user-input-driven outbound request function identified.	No SSRF path demonstrated; limited by discovered functionality.
Host header	Host and X-Forwarded-Host attempts did not alter relative reset links.	No host-header poisoning demonstrated.
Parameter pollution	Duplicate parameters resolved to the first value in the described tests.	No parameter-pollution bypass demonstrated.
Source/config paths	Common app.py, config.py, requirements, .env, .git and backup paths returned 404.	No disclosure from the tested candidate paths.

8. Tools and Evidence Referenced

The source notes refer to the following methods and artifacts. Exact tool versions and complete command logs were not included in the supplied Markdown files.

- HTTP request inspection and manipulation using Burp Suite/curl-style requests.
- TLS and response-header review.
- robots.txt inspection and route/path probing.
- Technology fingerprinting and static asset review.
- Authenticated crawling of patient pages and forms.
- Fuzzing and request-variation tests, with raw HTTP bodies and fuzz JSON reportedly saved under /tmp/opencode/tanclinic/.

The source material names evidence folders recon/ and evidence/ but does not attach those folders or the raw artifacts. This report therefore preserves the source's summarized outcomes without claiming independent evidence review.

9. Exclusions and De-duplication Against the NileCare Findings File

The following confirmed findings from the separate NileCare Security Findings document were intentionally excluded from this report's findings and conclusions:

- Stored XSS through attachment upload / uploaded HTML-polyglot behavior.
- IDOR / broken access control involving patient identity or report access.
- Information disclosure through the public doctors API.
- Client-controlled discount percentage / laboratory booking price manipulation.
- Password-reset token exposure, reuse or account takeover.
- Stored XSS in medical report fields and report-view rendering.

The AI report also contains other positive security observations—such as points-transfer race behavior, profile changes without re-authentication, admin account enumeration, session persistence after logout, cookie attributes and verbose/persistent flash messages. These were not converted into negative findings because they are described as successful weaknesses, not failed exploit attempts. They should be tracked in the main findings register or a separate supplemental findings report, after evidence and scope are reconciled.

10. Limitations and Confidence

- The report is based solely on two supplied Markdown files; the underlying screenshots, HTTP captures, fuzz outputs and evidence directory were not provided.
- The source notes contain multiple iterations and some inconsistent statements. For example, the admin route is described in one place as a decoy/stub and elsewhere as a genuine isolated login gate. This report uses the narrowest supported statement: no bypass or post-auth surface was demonstrated.
- The source notes report both 30,000 secret-key guesses and a later 38-candidate pass; neither resulted in a forged cookie according to the notes.
- The complete 100-entry robots.txt list was not present; only three example decoys and the live admin path were preserved.
- A 404 or rejected payload applies only to the exact path, request form and payloads tested; it does not establish that all related paths or variants are safe.
- The target was a shared lab instance and reportedly contained other testers' payloads. Results may have changed during the engagement.

11. Conclusion

Within the documented test scope, the team did not demonstrate RCE, SSTI, SQL injection, command injection, unsafe deserialization, SSRF, CORS exploitation, sensitive-file disclosure, directory listing, debug-console access, mass assignment, host-header poisoning, parameter-pollution bypass or an admin authentication bypass. Upload execution and traversal attempts described in this negative-testing subset were rejected or did not execute as code.

The most defensible overall conclusion is: **no additional exploit was validated for the negative test cases recorded in this report.** This should not be shortened to “the application is secure” or “the application is unhackable.” The separate findings report remains the authoritative location for confirmed vulnerabilities.

Appendix A — Candidate paths explicitly preserved in the source

Discovery files/paths: `/robots.txt`, `/static/uploads/attachments/`, `/console`.

Hidden admin route: `/admin_portal_temp_secure_9203`.

Admin alternate paths: `/admin`, `/administrator`, `/manage`, `/system`, `/wp-admin`.

Admin sub-route candidates: `/dashboard`, `/users`, `/settings`, `/logs`, `/api`, `/console`, `/export`, `/backup`, `/panel`.

Example robots.txt decoys: `/admin_private_95466`, `/api_dump_13280`, `/auth_restrict_1851`. The source states there were 100 candidate paths in total but does not include all names.

Source documents: Ai-REPORTS.md (uploaded) and NileCare_Security_Findings_Summary.md (uploaded).